



Stellar Cyber Log Relay Manager

Installing and using the log relay with your own syslog-ng, for a site with two Modular Sensors.

Log Relay Manager 1.1.67

Revised October 8, 2026

www.stellarcyber.ai

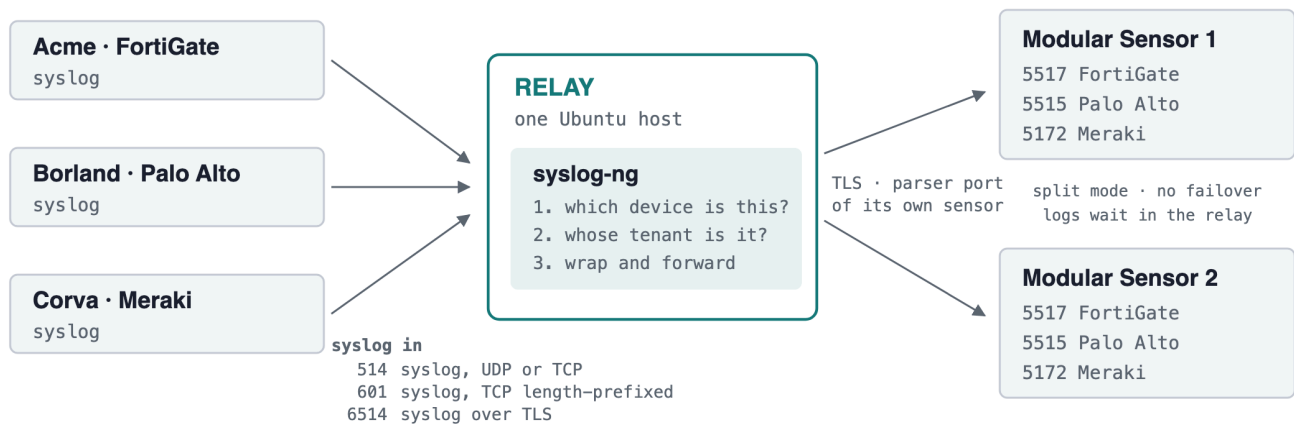
Contents

1. What the relay does
 2. Before you install: the host
 3. Before you install: syslog-ng, which is yours
 4. Before you install: from the Stellar Cyber platform
 5. Before you install: the network
 6. Installing the relay
 7. First sign-in: the agreement, then your two sensors
 8. Bringing in your tenants
 9. Adding a firewall
 10. Pointing the firewall at the relay, and proving it
 11. Certificates and TLS
 - The simplest way: the relay's own certificate authority
 - Using your own certificate authority
 - Which devices may connect over TLS
 12. Running it day to day
 - When a change causes an outage, and for how long
 - Updating the relay software
 - Moving log sources between sensors
 13. If logs do not arrive
- Appendix: the 45 parsers that accept the JSON wrapper

*Release 1.1.67 · 8 October 2026 · standard console · Ubuntu 24.04 or newer ·
syslog-ng 3.22 or newer*

This guide is for the administrator who will install and run the relay. It has three parts: what you have to have in place before the relay can be installed (the host, syslog-ng and a few facts from the Stellar Cyber platform), how to install the relay and connect it to your two sensors, and how it is used from day to day. It takes about an hour end to end.

1. What the relay does



Each log source is assigned to one sensor and stays there; while that sensor does not answer, its logs wait in the relay's buffer and are delivered when it returns. Nothing changes on the firewall except the address it sends syslog to — and the log itself is never modified, because the envelope travels beside it.

A Modular Sensor belongs to one tenant. To bring several customers' logs through the same sensors, something in front of them has to say which tenant each log belongs to. That is the relay's whole job.

Each firewall sends its ordinary syslog to one Ubuntu host. On that host, syslog-ng recognizes the device, wraps the log, unchanged, in a small Stellar Cyber envelope that carries the tenant ID, and forwards it over TLS to the parser's own port on the sensor. The sensor parses it exactly as if the firewall had sent it directly, and files it under the right tenant.

The Log Relay Manager is the web console that runs on the relay host. It does not replace syslog-ng and it does not install it. It configures the one file syslog-ng needs for this, keeps the tenant map, checks

every change with syslog-ng's own validator before applying it, and shows you whether logs are arriving and whether the sensors are taking them. syslog-ng itself, its updates and its system logging stay yours.

- **The log is not changed**

The relay wraps; it does not rewrite. What the sensor parses is the firewall's own line, byte for byte.

- **Tenant IDs are 32 hex characters**

Copy them from System → Tenants in Stellar Cyber. Anything else is silently ignored and the log lands in the sensor's own tenant.

- **One device is one log source**

A firewall serving several customers through VDOMs or virtual systems is still *one* log source, split by that field.

- **Nothing is guessed**

Every change is checked with `syslog-ng -s` before it is applied, and rolled back if syslog-ng refuses it.

2. Before you install: the host

One Ubuntu server, 24.04 or newer, that the firewalls can reach and that can reach both sensors. syslog-ng does the work on it; the Log Relay Manager is a small console beside it. Three things decide how big the host has to be:

- **Logs per second.** How many logs arrive at the busiest time of day, not the average. That sets the processor and the memory. Every log costs the relay the same whichever sensor it goes to, so the rate matters more than the number of firewalls or sensors.
- **TCP or UDP.** Over TCP or TLS every log is confirmed and none is lost on the way, but each firewall holds a connection open and the relay accepts 1,000 at a time on each port (a setting, if you have more devices than that). Over UDP there is no connection and no limit, but a burst larger than the receive buffer is lost before syslog-ng sees it; the Health page lists those drops.
- **A buffer for an outage.** While a sensor does not answer, its logs wait on the relay's disk and are delivered when it returns. The buffer syslog-ng starts with holds a few minutes. The disk, and the buffer setting, are sized by how long an outage you need to ride out.

Logs per second at the busiest time	vCPU	RAM	Disk	Buffer per sensor port
up to 1,000	2	2 GB	80 GB	10 GB (10737418240)
up to 2,000	2	4 GB	160 GB	20 GB (21474836480)
up to 5,000	4	8 GB	320 GB	50 GB (53687091200)

Starting points, measured on a test host, not guarantees: syslog-ng is your installation, and your log sizes, device mix and network decide the real figures. Start from the row above your expected rate (1,500 logs a

second is the 2,000 row), then read **Log relay services** for the first week — what was forwarded, what is waiting in the disk buffer, what was dropped — and adjust. The buffer column allows about four hours with one sensor unreachable; double it to cover both. The measurements behind the table are in the handbook under **Help** in the console, Part 2.

Setting the buffer. Configuration files → **syslog-ng relay configuration** → Edit, replace every `disk-buffer-size(536870912)` with the value from the table, press **Check**, then Save. syslog-ng reloads in place with no gap. A software update puts the default back, so repeat this after an update. The buffer files take no space until a sensor is actually unreachable.

3. Before you install: syslog-ng, which is yours

syslog-ng is your deployment. It comes from Ubuntu under its own license, you choose the version, and you keep the service, its updates and its system logging. The relay never installs, upgrades or removes it. On a host that does not have it yet, these three commands are the whole of it:

```
sudo apt update
sudo apt install syslog-ng-core syslog-ng-scl syslog-ng-mod-add-contextual-data
sudo systemctl enable --now syslog-ng
```

The third package is the one that matters most. `add-contextual-data` is how a log is matched to a tenant, and syslog-ng will not start with the relay's configuration without it. Then confirm four things:

- **The version is 3.22 or newer**

```
syslog-ng --version | head -1
```

Ubuntu 24.04 ships 4.3 and 26.04 ships 4.8, so a supported release is always new enough.

- **The module really loaded**

```
syslog-ng --version | grep add-contextual-data
```

It must print the module's name. Nothing printed means it is missing.

- **The service runs and starts at boot**

```
systemctl status syslog-ng --no-pager
```

 shows *active (running)* and *enabled*.

- **Nothing else holds the syslog ports**

```
sudo ss -lnup | grep 514
```

,

```
sudo ss -lnrp | grep 514
```

 and

```
sudo ss -lnrp | grep 601
```

should each print nothing. Ubuntu often has rsyslog installed; it does not listen on the network by default, but if it was turned on it owns 514 and syslog-ng cannot bind.

If anything is missing, the installer does not stop: `sudo bash install.sh --prereq syslog` prints this same list, checked against your host, step by step.

What the relay will and will not touch in your syslog-ng

It writes one file of its own, `/etc/syslog-ng/conf.d/stellar-relay.conf`; adds one line to `/etc/syslog-ng/syslog-ng.conf` (`@include "/etc/syslog-ng/conf.d/*.conf"`) if it is not already there; and keeps the tenant map at `/etc/syslog-ng/stellar-tenants.csv.src`. Nothing else in your syslog-ng configuration is read, rewritten or removed. Changes are applied by a reload, never a restart, so no log is dropped.

4. Before you install: from the Stellar Cyber platform

- **The address of each of your two sensors:** an IP address, or a host name, for each. A host name is looked up again every time syslog-ng loads, which is the reason to prefer one.
- **Syslog TLS turned on** in each sensor's profile (Log Forwarder → TLS). The relay sends over TLS by default; the setting must match on both sensors.
- **The tenant list.** In Stellar Cyber, open **System** → **Tenants**: the **Tenant Management** page (its address ends in `/system/admin/tenants`). Press **Export CSV**, the button beside **Create** at the top right of the table. That one file carries every customer's name, 32-character tenant ID and tenant group, and the console imports it exactly as it comes.
- **One catch-all tenant**, so that a device nobody has mapped yet lands somewhere harmless instead of in a real customer's data. Note its tenant ID.

5. Before you install: the network

Direction	Ports	Why
Firewalls → relay	udp 514, tcp 514, tcp 601, tcp 6514	Where the firewalls send syslog. The relay listens on all four and the device chooses. 514 is plain syslog, over UDP or TCP. 601 is syslog over TCP with each message length-prefixed, so two messages cannot run together or one be split (RFC 6587; FortiGate calls this <i>reliable</i> mode). 6514 is syslog over TLS (RFC 5425) — use it where the firewall supports it.
Relay → each sensor	5517 FortiGate, 5515 Palo Alto, 5172 Meraki, 5152 SonicWall, 5520 Sophos, 5519 Check Point, 5518 Cisco ASA	The parser port of each firewall type you use, on both sensors. Only the types you have are needed.
You → relay	tcp 8444	The console, from your own workstation.

In a cloud the security group needs the same rules. The console can also manage the host's own firewall once it is running.

6. Installing the relay

You receive the bundle `stellar-relay-1.1.67-signed.tar.gz`, about half a megabyte, with a small `stellar-relay-1.1.67-signed.tar.gz.sha1` beside it for checking the download; and its SHA-256 hash, sent separately.

Copy it to the relay host. From a Mac or a Linux machine, and from Windows 10 or 11 (which has `scp` built in — use PowerShell or Command Prompt), the command is the same. Replace `relay-ip-or-hostname` with the host's address, and `ubuntu` with the account you log in as:

```
scp stellar-relay-1.1.67-signed.tar.gz ubuntu@relay-ip-or-hostname:~
```

That is the whole command if you log in with a password. `-i` is only for a key file: if your host uses one, name yours in its place — it is whatever `.pem` or `.key` file you already use to reach that host.

```
scp -i <your identity key> stellar-relay-1.1.67-signed.tar.gz ubuntu@relay-ip-or-hostname:~
```

On an older Windows machine with PuTTY installed, the equivalent is `pscp stellar-relay-1.1.67-signed.tar.gz ubuntu@relay-ip-or-hostname:`. If you reach the host through a jump host or a bastion, add `-J user@bastion`.

Check the download arrived intact. A small `.sha1` file comes with the bundle, so this needs nothing from us and no comparing by eye. Run it where you downloaded the file and again on the relay host:

```
sha1sum -c stellar-relay-1.1.67-signed.tar.gz.sha1      # Linux
shasum -a 1 -c stellar-relay-1.1.67-signed.tar.gz.sha1 # macOS
```

Either prints `stellar-relay-1.1.67-signed.tar.gz: OK`. In Windows PowerShell, `Get-FileHash <file> -Algorithm SHA1` and compare with the line in the `.sha1` file.

And check the SHA-256 we sent you separately, which is a different question: the `.sha1` beside the file tells you the download is not corrupt, but anyone who can hand you a file can hand you a hash with it, so only a value that reached you by another route says anything about where the file came from. On your own machine: `sha256sum` on Linux, `shasum -a 256` on a Mac, `Get-FileHash <file> -Algorithm SHA256` in Windows PowerShell. (The relay settles the question properly for itself: it checks our signature before it unpacks anything, which no hash can do.) Then on the relay host, unpack it and run the installer:

```
# must print the SHA-256 you were given separately
sha256sum stellar-relay-1.1.67-signed.tar.gz

tar xzf stellar-relay-1.1.67-signed.tar.gz
cd stellar-relay-1.1.67
sudo bash install.sh
```

It detects syslog-ng on the host and asks for **one thing: the account you will sign in with.**

1. **An administrator username,** `admin` unless you give another. Letters, digits, dot, underscore or hyphen.
2. **Its password.** At least 12 characters, typed twice, never on the command line.

Everything else is asked in the console, at first sign-in: the license agreement — a short End User Notice — and where the logs go. Those are decisions worth seeing on a screen — an agreement is easier to read in a window than in a terminal, and a sensor address is easier to get right beside a live reachability check than at a prompt. Until the agreement is accepted the console does nothing else at all, so it cannot be skipped.

It does not ask for a tenant either. Tenants are brought in from the platform's own export and the catch-all is chosen in the console (sections 8 and 9), where you can see the list rather than typing a 32-character ID at a prompt.

It installs about 1.6 MB of files and prints the console's address — both of them, in fact: the one that works from inside your network and, when the host has a public address, the one that works from outside it. Only the inside one is reachable until your firewall or security group allows the console's port from your browser. Open it in a browser and sign in:

```
https://<syslog-ng-hostname>:8444
```

The certificate is self-signed, so the browser warns once. Section 11 shows how to replace it with one your certificate authority signs, from the console itself.

7. First sign-in: the agreement, then your two sensors

The license agreement comes first. Sign in and the console shows the **End User Notice & Terms** in full, with a tick box and **Accept and continue**. It is short: what this utility is for, and that your use of it is governed by the Stellar Cyber End User Agreement at <https://stellarcyber.ai/eua> — or by your organization's own executed master agreement with Stellar Cyber, which takes its place — under three terms specific to this product: the console is a control plane and never carries your logs; you review and approve every configuration it writes; and syslog-ng and the operating system stay under their own open-source licenses. The panel links that agreement, so it can be read before it is accepted. Nothing else in the console answers until it is accepted — not a page, not a setting — and the acceptance is recorded on the relay itself with the account that accepted it, the address it came from and the version. Only an administrator can accept it; a read-only account is told to fetch one. You are asked once per relay, not once per release.

Then: where do the logs go? The next panel asks for the Modular Sensor this relay forwards to, and a second one if you have two. An IPv4 address, or a host name this relay can resolve — a name is looked up again every time syslog-ng loads its configuration, so a sensor that moves needs no edit here. The same

panel carries one tick box, on by default: **keep a copy of every forwarded record on this host**. The console's "Last seen", its Log flow picture and its log explorer read that copy and show nothing without it; it is capped at 200 MB. Untick it only for a host that must not hold log content.

Pressing **Set up the relay** writes syslog-ng's configuration, validates it, and reloads syslog-ng in place — no gap, no log dropped. If syslog-ng refuses the result nothing is kept and the panel says why.

With that done the console opens normally, and the rest of this section is the **Sensors** page, where the same settings live from then on.

The console signs you out after an hour without activity: typing, clicking and scrolling count, a page refreshing itself does not. **Settings** → **Sign-in** changes the hour, between 5 minutes and 12 hours.

Open **Sensors** in the left menu. The page asks how many sensors this relay forwards to, because the two cases need different behaviour from the relay:

- **One sensor.** Everything goes to one address.
- **Several sensors, each log source picks its own.** This is your case. Two today; the page takes up to one hundred, added whenever the estate grows.

Choose **Several sensors, each log source picks its own** and enter both addresses. Each sensor gets a card of its own holding everything that is that sensor's: its address, whether it answers from this relay, its certificate and the CA that certificate is checked against, and how many log sources are pinned to it.

Check this sensor on a card tries the address *in the box*, so you can prove an address before you save it. Then the mode, which is the one decision on this page:

- **Split.** Your case today: you decide, per log source, which sensor its logs go to, exactly as you do now by pointing each firewall at one sensor or the other. Each log source carries a sensor (section 9), and moving one or many to the other sensor is one action (section 11). A sensor that stops answering keeps its records in the relay's disk buffer (512 MB per parser port) until it is back, so nothing is lost while one is down; the relay does not move them to the other sensor.
- **Failover.** Every log goes to sensor 1; sensor 2 takes over only while sensor 1 refuses. This mode needs syslog-ng's own failover() clause, and on syslog-ng 4.8.1 that clause makes every reload abort the process for as long as a sensor is unreachable, so the console refuses the mode on that version and says why. Use Split.

Adding a sensor later. + **Add another sensor** gives a new card, up to one hundred in all; nothing is written until the review window, and the new sensor then has everything sensor 1 has, including a CA of its own. Point log sources at it in section 11. **A sensor keeps its number for good:** if you later remove sensor 2 of three, sensor 3 is still sensor 3, because every log source names its sensor by number and renumbering would quietly move the ones pinned to a later sensor. A log source still pinned to a sensor that has been removed sends to sensor 1, and the console says how many are in that position before you remove it.

Connections to each sensor port. A Modular Sensor hands each TLS connection to one of its workers and keeps it there for the connection's life, so with the default of **one** connection per parser port everything the relay forwards to port 5517 — every FortiGate — lands on one worker however many the sensor has. The box under the transport opens 2, 4, 8 or 16 connections to each port instead, each from its own source port, so that many workers share it; records are divided evenly (one busy firewall spread over all of them) or by sending address (a device always on the same connection, its records in order). The port's disk buffer and memory window are divided between the connections, so no more disk or memory is used; four connections cost about 15% more processor per record than one, eight about 25%. Four is a good value for a sensor with four workers. Changing it is a reload, with no gap; lowering it later leaves whatever the removed connections still hold waiting until syslog-ng restarts, and the Log relay services page says so.

Seeing it work. The sensor's card and the **Reachability** tab count the open sessions per port (**5517 ×4**), with the relay's source ports on hover; the Overview writes **tls ×4** on the wire to the sensor. The **Delivery** tab shows, under a port with several sessions, each session's share of the records since Reset and the share over the last 30 seconds. In record mode the shares are close to equal (a session well below the others was usually reconnecting); in address mode a busy device keeps its session busy, so they differ by design. Press Reset, send the traffic, and read the split. A packet capture in the relay → sensor direction lists the same sessions as separate flows, numbered within their port. On the sensor host, `ss -tn state established '( sport = :5517 )'` lists one line per session from the relay's address, each with a different peer port; a sensor-side table that counts by source address and port shows them as one line, and one that is cumulative since the sensor started is dominated by the hours before the change, so read the increments over a minute rather than the totals.

Each sensor's certificate, chosen by name. A CA you upload for one sensor is kept under its name and offered on every other sensor's card, including a sensor you are adding and have not saved yet — so *the same CA as sensor 1* is one pick, and a different CA is one upload. A new sensor's choice is written with its address when you review and save the change.

A sensor that does not answer can still be saved. The check on each card is a real connection from the relay and nothing else depends on it; the relay queues records for a sensor until it answers. Removing a sensor takes its address and its CA out of the relay's settings for good, and the counters syslog-ng kept for it are forgotten once the change is applied.

Delivery and Volume. The **Delivery** tab is syslog-ng's own counters per sensor and per parser port: accepted, written, buffered on disk, dropped, and the logs a second arriving and leaving, with a **Reset** that counts from now and a find box. Beside **Waiting**, of $\sim N$ is how many logs that port can hold before syslog-ng drops the next one — its sessions, each with the memory window and the disk buffer of the running configuration, at the size of the logs the port really carries (a queued log costs about 650 bytes plus 2.9 times its own size on disk). The figure turns amber at half of that and red at 80%: that is the moment to act, before anything is dropped. The **Volume** tab is today against a baseline: after one complete day the baseline is that day; after a week it is the same weekday's average; adding or removing

a log source or a sensor starts it again, and the only question it answers is whether today is a significant change.

The mode can be changed later on this page; the sensor recorded on each log source is kept either way, so switching from failover to split brings the assignments into effect and switching back parks them. While neither sensor answers, logs wait in the relay's disk buffer and are delivered when one returns.

The button reads **Review the change to syslog-ng**. It shows the exact lines that would change, and only **Save and update** in that window writes anything; the result is validated first and rolled back if syslog-ng refuses it. The change is applied in place with no gap.

Two more things on that page:

- **Is the sensor operational?** A live check from the relay to each sensor's parser ports, repeated every 30 seconds while the page is open. It also shows the certificate each sensor presents. Green on both is what you want to see before adding any firewall.
- **The sensor's certificate.** On each sensor's own card: accept any certificate (what Stellar Cyber's documentation asks of senders while a sensor wears its self-signed default), or verify it against Stellar Cyber's CA, or against a CA you upload. A sensor with no CA of its own is checked against sensor 1's, so one CA for the whole estate stays one setting. The check above tells you whether verification would pass before you turn it on, so the switch never silently stops the flow.

8. Bringing in your tenants

Open **Log sources & tenants**, press **Import/Export** ▼ → **Import tenants from a CSV**, choose the file, and upload the tenant list you exported from the Tenant Management page in Stellar Cyber (System → Tenants → Export CSV). The columns may be in any order and the file may carry any number of columns the console does not need. It reads **Name**, **ID** and **Tenant Group**, creates each tenant group named in the file if it is not there yet, and puts each tenant in its group. Nothing in the relay's configuration changes at this point: this is the list of customers you are about to map devices to. A note can be added to any tenant afterwards, from its **Notes** menu; the platform's export does not carry one.

If you would rather prepare the file yourself, the same dialog offers **Minimum template (Stellar Cyber's field names)**: a file whose only columns are Name, ID, Tenant Group and an optional Description, exactly as the platform names them. Only those columns are ever read, by name, wherever they sit; any other column is ignored.

Re-upload the file whenever the list changes. Tenants already here keep their place; a row with no tenant group leaves that tenant where you put it.

9. Adding a firewall

Press **Add ▼** → **Log source** and choose the device type. The list holds the seven firewall types below and every parser in the appendix that reads the JSON wrapper on its own port. **Predefined options** are the default: for the seven, everything is already filled in - the port on the sensor, how the device is recognized, and which field, if any, separates customers on a shared appliance.

Firewall	Sensor port	Recognized by	Split between tenants by
FortiGate	5517	<code>devid</code> , the serial in every log	<code>vd</code> , the VDOM
Palo Alto Networks	5515	column 3 (Serial Number)	column 16 (Virtual System)
Cisco Meraki	5172	the address it sends from	never: one Meraki, one tenant
SonicWall	5152	<code>sn</code>	never
Sophos XG / XGS	5520	<code>device_id</code>	never
Check Point	5519	<code>origin</code>	never
Cisco ASA	5518	the address it sends from	never

Custom options. Choose **Custom options** to decide for yourself: recognize the device by the address it sends from or by any field in its logs, and send its logs to one tenant or split them by any field - for a firmware that writes its serial somewhere else, or to try another field. A device type from the appendix has no predefined options yet, so it is always custom: the port on the sensor is filled in for you, and **Find in a log...** shows the fields a real log from the device carries, which is how you find the one that names it. The choice is kept with the log source, and the relay refuses a predefined save that does not match the type's predefined options.

The dialog is three steps on one screen:

- 1. Log source device.** A name you will recognize, its type from the list, the public IP address or host name its logs come from, and, with two sensors, the **Sensor** its logs go to. For a Meraki or an ASA the address is required: it is how the relay recognizes the device. For the others, which the relay recognizes by a field in every log (a FortiGate's serial, a Palo Alto's serial column), it is optional: it is not written to syslog-ng, it narrows the sample-log search to this device, and left empty the relay records the address of the first log that arrives.
- 2. Which device this is.** The serial or ID the relay will look for in every log, named for you by the type. You do not have to know it: press **Find in a log...**, collect a few real logs from this device as they arrive at the relay, and click the value. With predefined options only column 3 can be picked for a Palo Alto and only `devid` for a FortiGate, so nothing can be chosen by mistake; with custom options any field can.
- 3. Whose logs these are.** One tenant, or, for a FortiGate or Palo Alto that serves several customers, one tenant per VDOM or virtual system with a default for any other value.

Save and review updates to syslog-ng shows the exact map rows and configuration lines that would change; only the confirmation in that window writes them, and syslog-ng reloads in place. Removing a log source is reviewed the same way, after its name is typed.

If two devices would share one identity. One device is one log source, so a new log source whose address or serial already belongs to another is not written into the map. It is saved anyway, **disabled** and marked *Config Conflict*, with the collision named on its row: which existing log source has that identity. Give one of the two its own identity and the row reads "conflict resolved"; **Enable** in its menu then writes it in. Nothing is ever merged or silently overwritten.

Many firewalls at once: **Import/Export** ▾ → **Import log sources from a CSV** takes a filled-in spreadsheet. Start from the empty template in that window, or from **Export log sources** in the toolbar, which writes every log source already on this relay - the one file this console exports. Every row is checked before anything is written; a row whose identity already belongs to another log source, or appears twice in the file, is kept disabled and marked *Import Conflict* in the same way. A row with `enabled=no` is imported disabled. The `options` column says `predefined` or `custom` for each row; left empty it is predefined wherever the device type has predefined options.

Not finished yet? Save it as a draft. **Save** (the button that does not update syslog-ng) on a form that still lacks something — the serial, the tenant — offers to keep it as a draft: nothing is written to syslog-ng, the row reads *draft*, and **Enable** later is the save that checks everything and writes it in.

10. Pointing the firewall at the relay, and proving it

On the firewall, set syslog to the relay's address on udp or tcp 514, tcp 601, or TLS 6514 where the device supports it. The device's menu has **Device setup**, which shows the vendor's own menu path with this relay's address filled in, and a **Watch for its logs now** button.

Three places say whether it is working, in order:

- 1. Overview → Log flow.** One row per log source, showing what it actually uses to reach the relay (TLS 6514, TCP 601, UDP 514) read off its live connection, lit when logs arrived in the last ten minutes.
- 2. Log sources & tenants → Last seen.** A green dot in front of the device's name means a log from it reached the relay in the last ten minutes. A firewall you added without an address now also shows the address its first log came from, marked "learned from its logs".
- 3. Sensors → Reachability.** Tests, from the relay itself, every parser port its log sources forward on, and says for each whether the sensor answered. On **Log relay services**, a port whose records were "accepted for this port and none delivered" is named under syslog-ng's status: that is the one line that means the sensor is not taking them.

Then search the Stellar Cyber platform for the firewall's events under the customer's tenant. Give the platform a couple of minutes to make new records searchable.

Problems, in one place. Under the version in the sidebar, on every page, a button reads *no problems* or *N problems*. It lists everything that needs resolution - a service not running, a port dropping or not delivering, a change saved but not applied, the copy of forwarded records off - each with a line to the page where it is resolved and an × to dismiss it (a dismissed one comes back when its words change). Nothing is listed before the first log source exists: a relay with nothing set up has nothing that can be wrong yet.

11. Certificates and TLS

The relay works with three certificates, each checked by someone different. Keep them apart: they are separate files with separate private keys, and replacing one never touches another.

Certificate	Who checks it	Where it is managed
The console's HTTPS certificate (port 8444)	The browsers that open the console	Settings → Security
syslog-ng's TLS certificate (port 6514)	The firewalls that send logs over TLS	Log relay services → Security
The sensor's certificate	The relay, when it forwards to the sensor	Sensors → Where it forwards , per sensor

The first two start out self-signed, so every browser warns and every firewall would need that exact certificate imported. A certificate authority removes both problems: it is installed once per browser and once per device, and every certificate it signs after that is trusted, including the ones that replace them when they expire.

The simplest way: the relay's own certificate authority

Settings → Certificate authority creates an authority on the relay and issues both certificates from it. Its private key is made on the relay, readable only by root, and never downloaded.

- 1. Create the authority**, once. Give it a name, which is shown as the issuer on everything it signs, and your organization's details. Creating it changes nothing by itself.
- 2. For the console.** Press **Download the CA for this browser** and install the file in your computer's or browser's trusted certificates *first*. Then press **Issue this console's HTTPS certificate...**, check that its names include the address you type in the browser, and issue it. The console starts using it at once; reload the page when it asks you to.
- 3. For the firewalls.** Press **Download the CA for the firewalls** and import that file on each FortiGate, Palo Alto or Meraki as a trusted CA. Then press **Issue the syslog-ng certificate...**, check that its names include the address the firewalls send to, and issue it. syslog-ng reloads to use it: no listener closes and nothing in flight is lost.
- 4. Point each firewall at the relay on port 6514 with TLS** (section 10), and watch for its logs.

Both downloads are the same file: the authority's certificate, never a private key. The certificates it issues last 825 days; renew one by issuing it again from the same authority, and nothing changes on the browsers or the devices, because they trust the authority, not the certificate.

A certificate must name the address it is reached at.

A browser compares its address bar with the certificate's names, and a firewall compares the address it sends to. A certificate that is right in every other way but does not carry that address is refused — and a firewall says nothing useful about it, it simply sends nothing. The issue form is prefilled with this host's names and addresses; add any other name or address the firewalls use.

Using your own certificate authority

If your organization already runs a certificate authority, press **We use our own authority instead: switch this one off** at the top of **Settings** → **Certificate authority**. The relay's authority then issues nothing and its certificate is no longer offered, so no browser or device is asked to trust it. Its files are kept, so it can be switched back on.

Install your certificates where they are used: the console's on **Settings** → **Security**, syslog-ng's on **Log relay services** → **Security**. Each card offers the same two ways:

- **Install a certificate and its key** — the certificate, its private key and, recommended, your CA's certificate, so the chain is checked before anything changes.
- **Or: create a signing request here** — the private key is made on the relay and never leaves it. Give the request to your CA, then install the certificate it returns.

Before anything changes, the console checks that the key belongs to the certificate, the certificate is within its dates and it chains to the CA you gave. The previous certificate is kept, and **Put the previous certificate back** restores it with one click.

Which devices may connect over TLS

Optional, and off by default: any device may connect on 6514, and the device's log source decides where its logs go. To require each firewall to present a certificate of its own, add the CA that issued the firewalls' certificates on **Log relay services** → **Security** → **Which devices may connect over TLS**. This applies to every device on 6514 at once, so turn it on only when every device sending over TLS can present a certificate; a Meraki cannot.

12. Running it day to day

When a change causes an outage, and for how long

In normal operation, nothing you do in this console stops logs. Every configuration change — adding or removing a log source, importing tenants, changing the catch-all, moving a source to the other sensor, changing a sensor address, replacing a certificate, editing a configuration file — is applied by a **reload**: syslog-ng re-reads its whole configuration in the same process, and the ports it is listening on are never closed. No log in flight is dropped and no firewall reconnects. None of these needs a maintenance window.

Exactly two things close the listeners, and you have to ask for both:

Action	What it costs	What is lost in the gap
Any configuration change (reload)	No gap. Same process, listeners stay open.	Nothing.
Restart on the Log relay services page	Listeners closed for about 0.04 s ; budget under a quarter second.	UDP syslog for that moment, with no retry. TCP and TLS sessions drop and the firewalls reconnect on their own. Logs already accepted are safe in the disk buffer.
A syslog-ng package upgrade	One restart, at the end. The old process forwards throughout the download and unpack.	The same as a restart, once. The console measures the real gap and shows it.
Turning the firewall on	No service is touched.	Logs from any device whose port has no rule. They never reach the relay, so they are lost rather than queued — check the rule list covers every listener first. The console refuses the most dangerous version of this on its own.

These are measurements, not estimates: a separate process connects to the listener every 5 milliseconds and counts refusals while the change is made. A reload gave no refusals at all in 1,135 attempts, with the process ID unchanged. Every confirm and review window in the console states the cost of that particular change before you commit it, and the **Log relay services** page lists all of them in one card, *What every change costs the log flow*.

One thing worth knowing: a configuration file **saved but not applied** costs nothing now, but it is on disk. If syslog-ng is restarted by something else — a reboot, or an upgrade — it will load the saved file. Apply it or put it back; the console shows a banner until you do.

- **Backups**

The whole configuration is archived every day at 03:15 to `/var/backups/stellar-relay/`, 30 days kept: the tenant maps and settings, the log source names and tenants, syslog-ng's TLS certificate and key, this console's HTTPS certificate and key, the certificate authority and its key, and every CA you uploaded. Each row says what it holds. **Every archive is encrypted** (AES-256) with a passphrase kept on this host; a restore here needs nothing from you, a restore on another host — or opening the file by hand — needs the passphrase, so **Show passphrase** on the Backups card once and keep it somewhere safe (showing it is recorded in the audit log; the passphrase itself never is).

Configuration → **Backups** → **Back up now** before any experiment, with a note saying what you are about to change. **Restore** on any row shows every file that would go back, takes a safety backup first, validates the result with syslog-ng and reloads in place; a restored console certificate is presented at once. The console's own secrets (its API token) and the audit log are archived but never rewound by a restore.

- **Changing a log source**

Its menu: Edit, Clone, Disable, Remove. **Disable** takes it out of the map but keeps its definition; **Enable** puts it back. When a device is still sending, both Disable and Remove warn in red that its logs will land in the catch-all tenant from then on.

- **Configuration files**

Every file the relay runs from, with a find box, an editor that checks syntax as you type, a history of every saved version with who saved it, and an Apply step. A save is validated by syslog-ng and rolled back if refused. If you have read the finding and disagree, **Save anyway, overriding the validator** keeps your text and records the refusal in the audit log; Apply still validates it again with the real program before anything is reloaded, and the confirmation says what an override costs for that file.

- **Log relay services**

Under the syslog-ng card's title, its status in words rather than a bare "active": running or not and since when, its process ID and version, the ports it listens on, whether a connection to a sensor is open, what it forwarded in the last five minutes and how much is waiting in the disk buffer, and one  line for each thing that needs a person: a restart by systemd, not enabled at boot, a sensor port that accepted logs and wrote none, a map or a file saved but not yet applied.

- **Health**

Whether syslog-ng is running, CPU and memory with a warning above 90%, and the service's process.

- **Software updates**

The **Log relay services** page can check Ubuntu for a newer syslog-ng and tells you: the version available, that installing it restarts syslog-ng once at the end — measured at about 0.04 s of closed listeners, budget under a quarter second — the `apt-get` command to run on the host, and the command that puts the previous version back if the new one misbehaves. It never installs or removes syslog-ng itself: that is yours to do, when you choose.

- **Accounts**

Admin accounts can change anything; read-only accounts can look at every page. Every change is written to an audit log with who made it and from where.

- **Updating the relay software**

When Stellar Cyber sends a newer bundle, upload it on **Settings** → **Update this console**. It is installed only if it carries Stellar Cyber's signature. It updates the management console only: syslog-ng is not touched and logs keep flowing. The steps are below, under *Updating the relay software*.

Updating the relay software

An update changes the management console only. It does not touch syslog-ng and causes no downtime.

The bundle updates the Log Relay Manager itself: the web console and its commands. syslog-ng is not upgraded, not reloaded and not restarted, and its configuration is not rewritten, so it keeps forwarding every log throughout with the same process. There is no maintenance window. While the console restarts, a few seconds, only the web page is unavailable. The checks that run before anything is installed also say whether the new version would ever write the syslog-ng configuration differently, and the update never applies such a difference on its own.

Stellar Cyber sends a newer bundle, `stellar-relay-<version>-signed.tar.gz`. It keeps every setting, log source, tenant and account, takes a backup first, and if the console does not answer afterwards it puts the previous version back by itself within 45 seconds.

How the relay knows a bundle is genuinely ours.

Every Stellar Cyber release is signed with a private key that never leaves Stellar Cyber, and your relay holds the matching public key, installed with the console itself. Before anything is unpacked, the relay checks the signature. A file that is not ours, or one that has had anything added to it or changed in it since we built it — a script added, an installer edited, a package swapped — fails that check and is refused, with the reason on screen. This is why you are not asked to compare a hash by eye: a hash only tells you a download was not corrupted, and whoever sends you a file can send you a hash with it. A signature cannot be forged without our key.

Once per relay, the first time: put two files on the host.

Signing arrives with this release, so a relay installed before it holds neither the public key nor the small program that reads it. It will refuse every bundle, including a genuine one, and say so. That is deliberate: what checks a file cannot come out of the file. We send you both files beside the bundle; copy them onto the relay once, and take them from us rather than out of the bundle — a bundle that supplied the key that judges it would simply be vouching for itself:

```
`` scp release_sig.py release-pubkey.pem ubuntu@relay-ip-or-hostname:~ ssh
ubuntu@relay-ip-or-hostname 'sudo install -o root -g root -m 644 ~/release_sig.py
~/release-pubkey.pem /opt/stellar-lab/common/' ``
```

Both are owned by the console package from this release on, so every update after that keeps them current and you never do this again. A relay installed fresh from this release already has them. **This first update is done from a shell** — the Update card described below arrives *with* this release, so a relay that predates it does not have one yet.

From the console (the simple way)

1. **Open Settings** → **Update this console** and press **Choose a release bundle....** Pick the `.tar.gz` we sent you, from your own computer — there is nothing to copy to the relay first.
2. **Read the line it gives you.** Green and ✓ *signed by Stellar Cyber* means the file is ours and untouched. Anything else is a refusal with the reason, and nothing has been unpacked; send us that reason.
3. **Press Update this console now**, type `update` to confirm, and wait. The page says what it is doing, goes quiet for a few seconds while the console restarts, and then says **Done** with the new version. Reload the page. Done offers two buttons: **Choose the next release...** for the update after this one, and **Roll back to v<previous>**, one step back. The bundle it installed is removed by itself.

If anything goes wrong, **Roll back** on the same card puts the previous version back. Every upload, check and install is in the audit log.

From a shell (the same checks, if you prefer)

1. **Copy the bundle to the relay host**, as in section 6:

```
scp stellar-relay-<version>-signed.tar.gz ubuntu@relay-ip-or-hostname:~
```

2. **Sign in to the relay host and check the hash.** It must print exactly the hash you were given:

```
sha256sum stellar-relay-<version>-signed.tar.gz
```

3. **Do a dry run.** It checks the bundle against this host's own map and running configuration, and changes nothing. There is no need to unpack the bundle:

A good answer starts with `ok signature: signed by Stellar Cyber` and continues with a line of `ok` per check, then `passed every check and would be installed ... Nothing was changed.` If any line does not say `ok`, stop there and send the output to Stellar Cyber.

```
sudo stellar-relay upgrade stellar-relay-<version>-signed.tar.gz --sha256 <hash> --dry-run
```

Keep `--sha256 <hash>` while the console you are upgrading *from* is older than 1.0.11: those versions require it and refuse without it. From 1.0.11 on it is optional, because the signature does that job; passing it anyway never hurts.

4. Run the update, the same command without `--dry-run`:

The last line should be `Upgraded: stellar-relay-monitor <old> -> <new>.` The console answers. Ubuntu's lines about kernels, services and containers before it are normal.

```
sudo stellar-relay upgrade stellar-relay-<version>-signed.tar.gz --sha256 <hash>
```

5. Check it. `stellar-relay version` prints the new version. In the browser, a strip says the console was updated: press **Reload the page**. The version under the host name in the left menu shows the new one, and the Overview still shows records forwarded.

An update only moves forward. A release older than the one running is refused, from the console (the moment it is uploaded) and from the shell alike. The one way back is a roll back, and it goes back one step: to the version the relay ran just before its last update. Settings → Update offers it as **Roll back**, or from the shell: `sudo stellar-relay rollback`. The backup taken before the update is in `/var/backups/stellar-relay/`. If you contact Stellar Cyber about an update, include the output of the upgrade command and of `stellar-relay version`.

Moving log sources between sensors

Open **Log sources & tenants** → **Log sources** (the flat list). With more than one sensor it has a **Sensor** column, a **Show** filter (every device, or only those on one sensor, with the count of each), and a tick box in front of every device. Tick the ones to move, one or fifty, choose the sensor in the bar above the list and press **Review and move**. The review window names the devices that move, the ones already there, and the exact map rows that change; **Move and update syslog-ng** writes them and reloads syslog-ng in place, and the next log from each device goes to its new sensor. A single device is moved the same way from its menu (*Send to sensor 2*), or by changing the Sensor box in its Edit dialog. Every move is in the audit log, and the History of changes on that page can undo it.

If the relay is in **failover** mode rather than **split**, the column is recorded but not in effect - every log goes to sensor 1 - and both the list and the review window say so, with a link to the Sensors page where the mode is set.

13. If logs do not arrive

What you see	Where to look
Nothing on the firewall's row in Log flow; "never seen" on Log sources & tenants	The logs are not reaching the relay. Check the firewall's syslog target and port, and the firewall-to-relay path (security group, on-premises firewall). On the relay: <code>sudo ss -lnup grep 514</code> shows syslog-ng listening.
Logs arrive but land in the catch-all tenant	The device was not recognized. On Log sources & tenants, open the device and press Find in a log...: the logs the relay is receiving show which value it actually carries, and whether the address matches.
Log relay services: "records accepted for this port and none delivered"	The relay cannot deliver to that sensor port. Check the relay-to-sensor path and that the sensor's TLS setting matches the relay's transport. The port test on Sensors → Reachability says reachable or not, from the relay itself.
A firewall set to send over TLS on 6514 sends nothing	Almost always the certificate: the firewall has not imported the CA (section 11), or the syslog-ng certificate does not name the address the firewall sends to. A FortiGate reports it as <code>Unknown CA</code> . Also check that tcp/6514 is open to the relay end to end.
Logs arrive on the platform under the sensor's own tenant	The tenant ID on that log source is not a valid 32-character ID. Open the source and pick the tenant from the imported list.
The console will not open after a certificate change	Settings → Security → Put the previous certificate back swaps the earlier pair back in at once. If the page itself cannot be reached any more, on the relay: <code>sudo stellar-relay-config console-cert-restore</code> , then <code>sudo systemctl restart relay-monitor</code> .
A change was refused	The message names the file, the line and what to change. Nothing was written: the previous version is still running.

Support: the backup archive (Configuration → Backups → Download) and the audit log (Audit log → Export) carry what support needs for a configuration question. Neither holds your customers' log data.

Appendix: the 45 parsers that accept the JSON wrapper

Each of these parsers reads the tenant from the Stellar Cyber JSON wrapper on its own port, so a log source of that type is sent to the port shown. The list is Appendix A of Stellar Cyber's *Multi-Tenant Log Ingestion Through a syslog-ng Log Relay*. The format is `syslog` for all 45; vendor and product are the values the wrapper carries to name the parser.

Parser	Sensor port	Vendor	Product
Accops	5526	<code>accops</code>	<code>accops</code>
Airgap Ransomware Kill Switch	5602	<code>airgap</code>	<code>ransomware_kill_switch</code>
Android	5605	<code>android</code>	<code>android</code>
Avaya Switch	5607	<code>avaya</code>	<code>switch</code>
BeyondTrust BeyondInsight	5621	<code>beyondtrust</code>	<code>beyondinsight</code>
Calyptix	5161	<code>calyptix</code>	<code>utm</code>

Parser	Sensor port	Vendor	Product
Centrify	5165	centrify	centrify_suite
Check Point Appliance	5174	checkpoint	appliance
Check Point EDR (Harmony)	5618	checkpoint	harmony
Cisco Firepower	5168	cisco	firepower
Cisco IKE	5176	cisco	ike
Cisco IronPort	5163	cisco	ironport
Cisco ISE	5157	cisco	ise
Cisco Meraki (legacy)	5172	cisco	meraki
Ericom ZTEdge	5603	ericom	ztedge
Extreme AirDefense	5612	extreme	air_defense
Extreme Network Access Control	5614	extreme	extremecontrol
F5 BIG-IP	5162	f5	big-ip
Fortinet FortiAnalyzer	5542	fortinet	fortianalyzer
Fortinet FortiGate	5517	fortinet	fortigate
Fortinet FortiMail	5616	fortinet	fortimail
Generic (BlackBerry Cylance)	5177	cylance	cylanceoptics
Juniper SRX	5173	juniper	srx
Juniper SSG	5516	juniper	ssg
Lepide	5606	lepide	data_security_platform
MonitorApp AI WAF	5613	monitorapp	ai_waf
NetApp	5608	netapp	netapp
NetIQ Access Manager	5167	netiq	access_manager
Ordr Connected Device Security	5622	ordr	connected_device_security
Palo Alto Networks Firewall	5515	palo_alto_networks	firewall
PIOLINK WEBFRONT-K	5617	piolink	webfront-k
Radware DefensePro	5619	radware	defensepro
SECUI MFD	5611	secui	mfd
SentinelOne Singularity Mobile	5623	sentinelone	singularity_mobile
ShareTech Firewall	5609	sharetech	utm_firewall
Snare Agent	5590	snare	snare_agent

Parser	Sensor port	Vendor	Product
SonicWall Firewall	5152	sonicwall	firewall
Sophos Firewall	5520	sophos	firewall
VMware UAG	5620	vmware	unified_access_gateway
VMware vCenter	5615	vmware	vcenter
Wazuh	5634	wazuh	wazuh_siem
Windows DNS Server	5599	windows	dns_server
Windows Event (NXLog)	5601	microsoft	windows
Windows System Security	5610	microsoft	system_security
Zenarmor	5604	sunny_valley_networks	zenarmor



STELLAR
CYBER®

www.stellarcyber.ai

California Office

2590 N. 1st St., Suite 360

San Jose, CA 95131